03 Marker-based cell-type annotation

Each nucleus label is derivedfrom a marker-versus-background contrast score:

score = mean logcounts of the class's present markers − mean logcounts of background genes

Background genes are those not present in any marker list. A marker counts as detected when its raw count is 2. Labels are assigned in the following order:

1. a neuron entry gate;
2. excitatory/inhibitory (EX/IN) polarity and subtype gates;
3. a neuron-vs-non-neuron competition;
4. a non-neuronal winner-take-all step with margin, absolute-floor and detection-fraction gates.

Clusters are not used for annotation. Nuclei that fail a gate are labelled `Neuron_unresolved` or `NonNeuron_unresolved`.

The EX/IN polarity uses the raw contrast difference `ex_score - in_score`.

```
suppressPackageStartupMessages({
  library(SingleCellExperiment)
  library(Matrix)
})
source(file.path("..", "scripts", "helpers.R"))
paths <- load_paths()
```

Parameters

```
marker_list_version <- "known_list_syms_v3_2026-04-10"
known_list_syms <- list(
  Neuron_core = c("SYT1", "RBFOX3", "STMN2", "TUBB3", "MAP2", "NEFL", "SNAP25", "SV2B",
   ↪  "SV2C"),
  OPCs = c("VCAN", "SMOC1", "OLIG1", "PDGFRA", "CSPG4", "GPR17", "OLIG2", "COL20A1",
   ↪  "PTPRZ1"),
  Oligodendrocytes = c("MOBP", "PLP1", "MBP", "MOG", "MAG", "SLC44A1", "CNP", "MYRF",
   ↪  "OPALIN"),
  Astrocytes = c("ALDH1L1", "ETNPPL", "GJA1", "CLU", "AQP4", "SLC1A3", "SLC1A2", "GFAP"),
  Microglia = c("CSF1R", "TMEM119", "TREM2", "LST1", "P2RY12", "TYROBP", "CX3CR1",
                "APBB1IP", "SLCO2B1", "TBXAS1"),
  Excitatory_neurons = c("SLC17A7", "SLC17A6", "SATB2", "TBR1", "EMX1", "CUX2", "RORB",
                         "TLE4", "SEMA3E", "GPC3", "SLC17A8"),
  Inhibitory_neurons = c("GAD1", "GAD2", "SLC6A1", "SLC32A1", "DLX1", "DLX2", "LHX6",
                         "PVALB", "SST", "VIP", "RELN", "NDNF", "CALB2", "GRIK1", "CNR1",
                         "ADARB2", "NOS1", "SOX6"),
  Endothelial = c("KDR", "FLT1", "PECAM1", "VWF", "EMCN", "RAMP2", "ESAM", "CLDN5",
   ↪  "ROBO4"),
  Pericytes = c("PDGFRB", "RGS5", "MCAM", "ABCC9", "NOTCH3", "KCNJ8"))

# Scoring
min_genes     <- 2L
det_min_count <- 2L    # raw count >= 2 defines detection
```

```
bg_min_genes  <- 50L   # background gene threshold

# Neuron entry gate
core_det_min   <- 5L
core_score_min <- 0.35

# EX/IN subtype evidence
ex_transporters <- c("SLC17A7", "SLC17A6", "SLC17A8")
ex_tf           <- c("EMX1", "TBR1", "TLE4", "SATB2", "CUX2", "RORB")
in_transporters <- c("GAD1", "GAD2", "SLC32A1", "SLC6A1", "ADARB2", "PVALB", "SST",
                     "VIP", "RELN", "NDNF", "CALB2", "CNR1", "GRIK1")
in_tf           <- c("DLX1", "DLX2", "LHX6", "SOX6")

# EX/IN gates
margin_req_both     <- 0.25
margin_req_one      <- 0.10
subtype_score_floor <- 0.25
subtype_det_min     <- 1L

# Non-neuronal competition
nn_margin_cut <- 0.10
nn_abs_floor  <- 0.25
det_frac_cut  <- c(Excitatory_neurons = 0.30, Inhibitory_neurons = 0.30, Astrocytes =
 ↪  0.25,
                   OPCs = 0.30, Oligodendrocytes = 0.30, Microglia = 0.30,
                   Endothelial = 0.25, Pericytes = 0.25, Neuron_core = 0.30)
det_frac_fallback <- 0.10
competition_exclude <- c("Neuron_core", "Excitatory_neurons", "Inhibitory_neurons")

resolved_labels   <- c("Excitatory_neurons", "Inhibitory_neurons", "Astrocytes", "OPCs",
                       "Oligodendrocytes", "Microglia", "Endothelial", "Pericytes")
unresolved_labels <- c("Neuron_unresolved", "NonNeuron_unresolved")
```

```
sce  <- readRDS(file.path(paths$work_dir, "02_clustered.rds"))
mats <- symbol_matrices(sce, "symbol")
counts_sym <- mats$counts
log_sym    <- mats$logcounts
cells <- colnames(sce)
n <- length(cells)
```

Marker-contrast scores

```
all_genes  <- rownames(counts_sym)
classes    <- names(known_list_syms)
any_marker <- intersect(unique(unlist(known_list_syms)), all_genes)

score    <- matrix(NA_real_, n, length(classes), dimnames = list(cells, classes))
det      <- matrix(0L,       n, length(classes), dimnames = list(cells, classes))
det_frac <- matrix(0,        n, length(classes), dimnames = list(cells, classes))

for (ct in classes) {
  g_pos <- intersect(known_list_syms[[ct]], all_genes)
  if (length(g_pos) < min_genes) next
```

```
  g_bg <- setdiff(setdiff(all_genes, any_marker), g_pos)
  if (length(g_bg) < bg_min_genes) g_bg <- setdiff(all_genes, g_pos)
  det_ct <- Matrix::colSums(counts_sym[g_pos, , drop = FALSE] >= det_min_count)
  det[, ct]      <- as.integer(det_ct)
  det_frac[, ct] <- as.numeric(det_ct) / length(g_pos)
  score[, ct] <- as.numeric(Matrix::colMeans(log_sym[g_pos, , drop = FALSE]) -
                            Matrix::colMeans(log_sym[g_bg, , drop = FALSE]))
}
```

Neuron entry gate and EX/IN assignment

A nucleus is considered neuronal - if it passes the neuron-core gate (5 core markers detected and core score 0.35), or - if it has both a transporter and a transcription-factor hit for the excitatory or inhibitory set.

```
detect_over <- function(syms) {
  ids <- intersect(unique(syms), rownames(counts_sym))
  if (length(ids) == 0L) return(rep.int(0L, n))
  as.integer(Matrix::colSums(counts_sym[ids, , drop = FALSE] >= det_min_count))
}
has_ex_ev <- detect_over(ex_transporters) >= 1L & detect_over(ex_tf) >= 1L
has_in_ev <- detect_over(in_transporters) >= 1L & detect_over(in_tf) >= 1L

core_score <- score[, "Neuron_core"]
core_ok <- det[, "Neuron_core"] >= core_det_min & is.finite(core_score) &
           core_score >= core_score_min
is_neuron_core <- core_ok | has_ex_ev | has_in_ev

coarse <- rep("Unk", n)
coarse[is_neuron_core] <- "Neuron_unresolved"

ex_score <- score[, "Excitatory_neurons"]
in_score <- score[, "Inhibitory_neurons"]
exin_margin <- rep(NA_real_, n)
nc <- which(is_neuron_core)

if (length(nc) > 0L) {
  d    <- ex_score[nc] - in_score[nc]
  pred <- ifelse(d >= 0, "Excitatory_neurons", "Inhibitory_neurons")
  m    <- abs(d)
  is_ex <- pred == "Excitatory_neurons"
  det_pred   <- ifelse(is_ex, det[nc, "Excitatory_neurons"], det[nc,
 ↪  "Inhibitory_neurons"])
  score_pred <- ifelse(is_ex, ex_score[nc], in_score[nc])
  ev_pred    <- ifelse(is_ex, has_ex_ev[nc], has_in_ev[nc])
  req <- ifelse(has_ex_ev[nc] & has_in_ev[nc], margin_req_both, margin_req_one)
  ok <- (is.finite(m) & m >= req) &
        (det_pred >= subtype_det_min) &
        ev_pred &
        (is.finite(score_pred) & score_pred >= subtype_score_floor)
  coarse[nc[ok]]  <- pred[ok]
  coarse[nc[!ok]] <- "Neuron_unresolved"
  exin_margin[nc] <- m
}
```

Neuron-vs-non-neuron competition and non-neuronal gates

A neuron candidate keeps its neuronal call only if its best neuronal score (neuron core, EX or IN) beats the best non-neuronal score. Every other nucleus goes through the non-neuronal winner-take-all step.

```
comp_cols <- setdiff(colnames(score), competition_exclude)
score_comp <- score[, comp_cols, drop = FALSE]
score_comp[!is.finite(score_comp)] <- -Inf

top1_all <- score_comp[cbind(seq_len(n), max.col(score_comp, ties.method = "first"))]
top1_all[!is.finite(top1_all)] <- NA_real_
best_neuron <- suppressWarnings(pmax(score[, "Neuron_core"], ex_score, in_score, na.rm =
 ↪  TRUE))
best_neuron[!is.finite(best_neuron)] <- NA_real_
neuron_wins <- is_neuron_core & is.finite(best_neuron) & is.finite(top1_all) &
               best_neuron > top1_all

remain <- !neuron_wins
coarse[remain] <- "NonNeuron_unresolved"

ridx <- which(remain)
nn <- data.frame(cell_id = cells, nonneuron_top1_class = NA_character_,
                 nonneuron_top1_score = NA_real_, nonneuron_score_margin = NA_real_,
                 nonneuron_predicted_detection_fraction = NA_real_,
                 nonneuron_margin_pass = NA, stringsAsFactors = FALSE)
if (length(ridx) > 0L) {
  sc  <- score_comp[ridx, , drop = FALSE]
  i1  <- max.col(sc, ties.method = "first")
  m1  <- sc[cbind(seq_along(ridx), i1)]
  tmp <- sc; tmp[cbind(seq_along(ridx), i1)] <- -Inf
  m2  <- tmp[cbind(seq_along(ridx), max.col(tmp, ties.method = "first"))]
  m2[!is.finite(m2)] <- NA_real_
  marg <- m1 - m2
  pred_ct <- colnames(sc)[i1]
  detf1 <- det_frac[ridx, comp_cols, drop = FALSE][cbind(seq_along(ridx), i1)]
  kf <- ifelse(pred_ct %in% names(det_frac_cut), det_frac_cut[pred_ct],
 ↪  det_frac_fallback)
  pass_margin <- is.finite(marg) & marg >= nn_margin_cut
  pass_abs    <- is.finite(m1) & m1 >= nn_abs_floor
  pass_detf   <- detf1 >= kf
  ok <- pass_margin & pass_abs & pass_detf
  final_ct <- pred_ct
  final_ct[!ok] <- "NonNeuron_unresolved"
  coarse[ridx] <- final_ct

  m1[!is.finite(m1)] <- NA_real_
  nn$nonneuron_top1_class[ridx]   <- pred_ct
  nn$nonneuron_top1_score[ridx]   <- m1
  nn$nonneuron_score_margin[ridx] <- marg
  nn$nonneuron_predicted_detection_fraction[ridx] <- detf1
  nn$nonneuron_margin_pass[ridx]  <- pass_margin
}

stopifnot(!any(coarse == "Unk"), all(coarse %in% c(resolved_labels, unresolved_labels)))
table(coarse)
```

```
## coarse
##           Astrocytes          Endothelial   Excitatory_neurons
##                   52                    4                  318
##   Inhibitory_neurons            Microglia    Neuron_unresolved
##                  128                   37                  175
## NonNeuron_unresolved     Oligodendrocytes                 OPCs
##                   23                  226                   82
##            Pericytes
##                    5
```

Output

```
per_cell <- data.frame(
  cell_id = cells,
  final_celltype_label = coarse,
  neuron_core_marker_contrast = as.numeric(core_score),
  exin_margin = exin_margin,
  nn[, -1],
  stringsAsFactors = FALSE)
for (ct in classes) per_cell[[paste0("marker_contrast_", ct)]] <- score[, ct]

sce$final_celltype_label <- coarse

# Verify against the deposited annotation
dep <- read.delim(file.path(paths$zenodo_root, "metadata",
 ↪  "final_cell_type_annotations.tsv.gz"),
                  stringsAsFactors = FALSE)
stopifnot(identical(coarse, dep$final_cell_type[match(cells, dep$cell_id)]))

saveRDS(sce, file.path(paths$work_dir, "03_annotated.rds"))
saveRDS(list(marker_list_version = marker_list_version, known_list_syms =
 ↪  known_list_syms,
             exin_evidence = list(ex_transporters = ex_transporters, ex_tf = ex_tf,
                                  in_transporters = in_transporters, in_tf = in_tf)),
        file.path(paths$work_dir, "03_marker_lists.rds"))

save_tsv(per_cell, file.path(paths$work_dir, "03_annotation_per_cell.tsv"))

sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.5.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.
 ↪  framework/Versions/A/libBLAS.dylib
## LAPACK:
 ↪  /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
 ↪  LAPACK version 3.12.1
##
## locale:
## [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
##
## time zone: America/New_York
```

```
## tzcode source: internal
##
## attached base packages:
## [1] stats4    stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] igraph_2.2.1                scater_1.38.0
##  [3] ggplot2_4.0.1               scran_1.38.0
##  [5] Matrix_1.7-4                scuttle_1.20.0
##  [7] SingleCellExperiment_1.32.0 SummarizedExperiment_1.40.0
##  [9] Biobase_2.70.0              GenomicRanges_1.62.1
## [11] Seqinfo_1.0.0               IRanges_2.44.0
## [13] S4Vectors_0.48.0            BiocGenerics_0.56.0
## [15] generics_0.1.4              MatrixGenerics_1.22.0
## [17] matrixStats_1.5.0
##
## loaded via a namespace (and not attached):
##  [1] beeswarm_0.4.0      gtable_0.3.6        xfun_0.55
##  [4] ggrepel_0.9.6       lattice_0.22-7      vctrs_0.7.1
##  [7] tools_4.5.2         parallel_4.5.2      tibble_3.3.1
## [10] cluster_2.1.8.2     pkgconfig_2.0.3     BiocNeighbors_2.4.0
## [13] RColorBrewer_1.1-3  S7_0.2.1            dqrng_0.4.1
## [16] lifecycle_1.0.5     compiler_4.5.2      farver_2.1.2
## [19] statmod_1.5.1       tinytex_0.58        bluster_1.20.0
## [22] codetools_0.2-20    vipor_0.4.7         htmltools_0.5.9
## [25] yaml_2.3.12         pillar_1.11.1       BiocParallel_1.44.0
## [28] DelayedArray_0.36.0 limma_3.66.0        viridis_0.6.5
## [31] abind_1.4-8         tidyselect_1.2.1    rsvd_1.0.5
## [34] locfit_1.5-9.12     metapod_1.18.0      digest_0.6.39
## [37] BiocSingular_1.26.0 dplyr_1.2.0         fastmap_1.2.0
## [40] grid_4.5.2          cli_3.6.5           SparseArray_1.10.8
## [43] magrittr_2.0.4      S4Arrays_1.10.1     withr_3.0.2
## [46] edgeR_4.8.2         scales_1.4.0        ggbeeswarm_0.7.3
## [49] rmarkdown_2.30      XVector_0.50.0      otel_0.2.0
## [52] gridExtra_2.3       ScaledMatrix_1.18.0 beachmat_2.26.0
## [55] evaluate_1.0.5      knitr_1.51          viridisLite_0.4.2
## [58] irlba_2.3.7         rlang_1.1.7         Rcpp_1.1.1
## [61] glue_1.8.0          R6_2.6.1
```